# Chapter 13. Indexes and Constraints

Because the focus of this book is on programming techniques, the first 12 chapters concentrated on elements of the SQL language that you can use to craft powerful `select`, `insert`, `update`, and `delete` statements. However, other database features *indirectly* affect the code you write. This chapter focuses on two of those features: indexes and constraints.

## Indexes

When you insert a row into a table, the database server does not attempt to put the data in any particular location within the table. For example, if you add a row to the `customer` table, the server doesn't place the row in numeric order via the `customer_id` column or in alphabetical order via the `last_name` column. Instead, the server simply places the data in the next available location within the file (the server maintains a list of free space for each table). When you query the `customer` table, therefore, the server will need to inspect every row of the table to answer the query. For example, let's say that you issue the following query:

```
mysql> SELECT first_name, last_name
    -> FROM customer
    -> WHERE last_name LIKE 'Y%';
+------------+-----------+
| first_name | last_name |
+------------+-----------+
| LUIS       | YANEZ     |
| MARVIN     | YEE       |
| CYNTHIA    | YOUNG     |
+------------+-----------+
3 rows in set (0.09 sec)
```

To find all customers whose last name begins with *Y*, the server must visit each row in the `customer` table and inspect the contents of the `last_name` column; if the last name

begins with *Y*, then the row is added to the result set. This type of access is known as a *table scan*.

While this method works fine for a table with only three rows, imagine how long it might take to answer the query if the table contains three million rows. At some number of rows larger than three and smaller than three million, a line is crossed where the server cannot answer the query within a reasonable amount of time without additional help. This help comes in the form of one or more *indexes* on the `customer` table.

Even if you have never heard of a database index, you are certainly aware of what an index is (e.g., this book has one). An index is simply a mechanism for finding a specific item within a resource. Each technical publication, for example, includes an index at the end that allows you to locate a specific word or phrase within the publication. The index lists these words and phrases in alphabetical order, allowing the reader to move quickly to a particular letter within the index, find the desired entry, and then find the page or pages on which the word or phrase may be found.

In the same way that a person uses an index to find words within a publication, a database server uses indexes to locate rows in a table. Indexes are special tables that, unlike normal data tables, *are* kept in a specific order. Instead of containing *all* of the data about an entity, however, an index contains only the column (or columns) used to locate rows in the data table, along with information describing where the rows are physically located. Therefore, the role of indexes is to facilitate the retrieval of a subset of a table's rows and columns *without* the need to inspect every row in the table.

## Index Creation

Returning to the `customer` table, you might decide to add an index on the `email` column to speed up any queries that specify a value for this column, as well as any `update` or `delete` operations that specify a customer's email address. Here's how you can add such an index to a MySQL database:

```
mysql> ALTER TABLE customer
    -> ADD INDEX idx_email (email);
Query OK, 0 rows affected (1.87 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

This statement creates an index (a B-tree index to be precise, but more on this shortly) on the `customer.email` column; furthermore, the index is given the name `idx_email`. With the index in place, the query optimizer (which we discussed in Chapter 3) can

choose to use the index if it is deemed beneficial to do so. If there is more than one index on a table, the optimizer must decide which index will be the most beneficial for a particular SQL statement.

---

NOTE

MySQL treats indexes as optional components of a table, which is why in earlier versions you would use the `alter table` command to add or remove an index. Other database servers, including SQL Server and Oracle Database, treat indexes as independent schema objects. For both SQL Server and Oracle, therefore, you would generate an index using the `create index` command, as in:

```
CREATE INDEX idx_email
ON customer (email);
```

As of MySQL version 5, a `create index` command is available, although it is mapped to the `alter table` command. You must still use the `alter table` command to create primary key indexes, however.

---

All database servers allow you to look at the available indexes. MySQL users can use the `show` command to see all of the indexes on a specific table, as in:

```
mysql> SHOW INDEX FROM customer \G;
*************************** 1. row ***************************
        Table: customer
   Non_unique: 0
     Key_name: PRIMARY
  Seq_in_index: 1
  Column_name: customer_id
    Collation: A
  Cardinality: 599
     Sub_part: NULL
       Packed: NULL
         Null:
   Index_type: BTREE
...
*************************** 2. row ***************************
        Table: customer
   Non_unique: 1
     Key_name: idx_fk_store_id
  Seq_in_index: 1
  Column_name: store_id
    Collation: A
```

```
  Cardinality: 2
     Sub_part: NULL
       Packed: NULL
         Null:
   Index_type: BTREE
...
*************************** 3. row ***************************
        Table: customer
   Non_unique: 1
     Key_name: idx_fk_address_id
  Seq_in_index: 1
  Column_name: address_id
    Collation: A
  Cardinality: 599
     Sub_part: NULL
       Packed: NULL
         Null:
   Index_type: BTREE
...
*************************** 4. row ***************************
        Table: customer
   Non_unique: 1
     Key_name: idx_last_name
  Seq_in_index: 1
  Column_name: last_name
    Collation: A
  Cardinality: 599
     Sub_part: NULL
       Packed: NULL
         Null:
   Index_type: BTREE
...
*************************** 5. row ***************************
        Table: customer
   Non_unique: 1
     Key_name: idx_email
  Seq_in_index: 1
  Column_name: email
    Collation: A
  Cardinality: 599
     Sub_part: NULL
       Packed: NULL
         Null: YES
   Index_type: BTREE
...
5 rows in set (0.06 sec)
```

The output shows that there are five indexes on the `customer` table: one on the `customer_id` column called `PRIMARY`, and four others on the `store_id`, `address_id`, `last_name`, and `email` columns. If you are wondering where these indexes came from, I created the index on the `email` column, and the rest were installed as part of the sample Sakila database. Here's the statement used to create the table:

```
CREATE TABLE customer (
  customer_id SMALLINT UNSIGNED NOT NULL AUTO_INCREMENT,
  store_id TINYINT UNSIGNED NOT NULL,
  first_name VARCHAR(45) NOT NULL,
  last_name VARCHAR(45) NOT NULL,
  email VARCHAR(50) DEFAULT NULL,
  address_id SMALLINT UNSIGNED NOT NULL,
  active BOOLEAN NOT NULL DEFAULT TRUE,
  create_date DATETIME NOT NULL,
  last_update TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY  (customer_id),
  KEY idx_fk_store_id (store_id),
  KEY idx_fk_address_id (address_id),
  KEY idx_last_name (last_name),
  ...
```

When the table was created, the MySQL server automatically generated an index on the primary key column, which in this case is `customer_id`, and gave the index the name `PRIMARY`. This is a special type of index used with a primary key constraint, but I will cover constraints later in this chapter.

If, after creating an index, you decide that the index is not proving useful, you can remove it via the following:

```
mysql> ALTER TABLE customer
    -> DROP INDEX idx_email;
Query OK, 0 rows affected (0.50 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

NOTE

SQL Server and Oracle Database users must use the `drop index` command to remove an index, as in:

```
DROP INDEX idx_email; (Oracle)

DROP INDEX idx_email ON customer; (SQL Server)
```

MySQL now also supports the `drop index` command, although it is also mapped to the `alter table` command.

---

## Unique indexes

When designing a database, it is important to consider which columns are allowed to contain duplicate data and which are not. For example, it is allowable to have two customers named John Smith in the `customer` table since each row will have a different identifier ( `customer_id` ), email, and address to help tell them apart. You would not, however, want to allow two different customers to have the same email address. You can enforce a rule against duplicate values by creating a *unique index* on the `customer.email` column.

A unique index plays multiple roles; along with providing all the benefits of a regular index, it also serves as a mechanism for disallowing duplicate values in the indexed column. Whenever a row is inserted or when the indexed column is modified, the database server checks the unique index to see whether the value already exists in another row in the table. Here's how you would create a unique index on the `customer.email` column:

```
mysql> ALTER TABLE customer
    -> ADD UNIQUE idx_email (email);
Query OK, 0 rows affected (0.64 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

NOTE

SQL Server and Oracle Database users need only add the `unique` keyword when creating an index, as in:

```
CREATE UNIQUE INDEX idx_email
ON customer (email);
```

---

With the index in place, you will receive an error if you try to add a new customer with an email address that already exists:

```
mysql> INSERT INTO customer
    ->  (store_id, first_name, last_name, email, address_id, active)
    -> VALUES
    ->  (1,'ALAN','KAHN', 'ALAN.KAHN@sakilacustomer.org', 394, 1);
ERROR 1062 (23000): Duplicate entry 'ALAN.KAHN@sakilacustomer.org'
 for key 'idx_email'
```

You should not build unique indexes on your primary key column(s), since the server already checks uniqueness for primary key values. You may, however, create more than one unique index on the same table if you feel that it is warranted.

### Multicolumn indexes

Along with the single-column indexes demonstrated thus far, you may also build indexes that span multiple columns. If, for example, you find yourself searching for customers by first *and* last names, you can build an index on *both* columns together, as in:

```
mysql> ALTER TABLE customer
    -> ADD INDEX idx_full_name (last_name, first_name);
Query OK, 0 rows affected (0.35 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

This index will be useful for queries that specify the first and last names or just the last name, but it would not be useful for queries that specify only the customer's first name. To understand why, consider how you would find a person's phone number; if you know the person's first and last names, you can use a phone book to find the number quickly, since a phone book is organized by last name and then by first name. If you know only

the person's first name, you would need to scan every entry in the phone book to find all the entries with the specified first name.

When building multiple-column indexes, therefore, you should think carefully about which column to list first, which column to list second, and so on, to help make the index as useful as possible. Keep in mind, however, that there is nothing stopping you from building multiple indexes using the same set of columns but in a different order if you feel that it is needed to ensure adequate response time.

## Types of Indexes

Indexing is a powerful tool, but since there are many different types of data, a single indexing strategy doesn't always do the job. The following sections illustrate the different types of indexing available from various servers.

### B-tree indexes

All the indexes shown thus far are *balanced-tree indexes*, which are more commonly known as *B-tree indexes*. MySQL, Oracle Database, and SQL Server all default to B-tree indexing, so you will get a B-tree index unless you explicitly ask for another type. As you might expect, B-tree indexes are organized as trees, with one or more levels of *branch nodes* leading to a single level of *leaf nodes*. Branch nodes are used for navigating the tree, while leaf nodes hold the actual values and location information. For example, a B-tree index built on the `customer.last_name` column might look something like Figure 13-1.

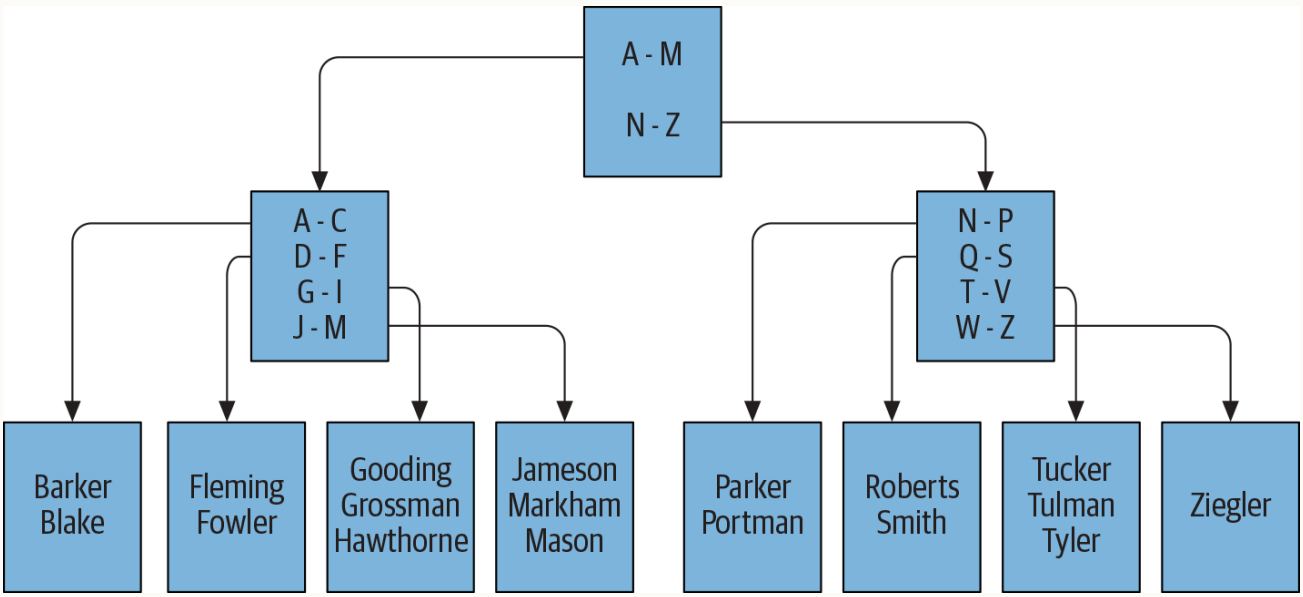



*Figure 13-1. B-tree example*

If you were to issue a query to retrieve all customers whose last name starts with *G*, the server would look at the top branch node (called the *root node*) and follow the link to the branch node that handles last names beginning with *A* through *M*. This branch node would, in turn, direct the server to a leaf node containing last names beginning with *G* through *I*. The server then starts reading the values in the leaf node until it encounters a value that doesn't begin with *G* (which, in this case, is Hawthorne).

As rows are inserted, updated, and deleted from the `customer` table, the server will attempt to keep the tree balanced so that there aren't far more branch/leaf nodes on one side of the root node than the other. The server can add or remove branch nodes to redistribute the values more evenly and can even add or remove an entire level of branch nodes. By keeping the tree balanced, the server is able to traverse quickly to the leaf nodes to find the desired values without having to navigate through many levels of branch nodes.

### Bitmap indexes

Although B-tree indexes are great at handling columns that contain many different values, such as a customer's first/last names, they can become unwieldy when built on a column that allows only a small number of values. For example, you may decide to generate an index on the `customer.active` column so that you can quickly retrieve all active or inactive accounts. Because there are only two different values (stored as 1 for active and 0 for inactive), however, and because there are far more active customers, it can be difficult to maintain a balanced B-tree index as the number of customers grows.

For columns that contain only a small number of values across a large number of rows (known as *low-cardinality* data), a different indexing strategy is needed. To handle this situation more efficiently, Oracle Database includes *bitmap indexes*, which generate a bitmap for each value stored in the column. If you were to build a bitmap index on the `customer.active` column, the index would maintain two bitmaps: one for the value 0 and another for the value 1. When you write a query to retrieve all inactive customers, the database server can traverse the 0 bitmap and quickly retrieve the desired rows.

Bitmap indexes are a nice, compact indexing solution for low-cardinality data, but this indexing strategy breaks down if the number of values stored in the column climbs too high in relation to the number of rows (known as *high-cardinality* data), since the server would need to maintain too many bitmaps. For example, you would never build a bitmap index on your primary key column, since this represents the highest possible cardinality (a different value for every row).

Oracle users can generate bitmap indexes by simply adding the `bitmap` keyword to the `create index` statement, as in:

```
CREATE BITMAP INDEX idx_active ON customer (active);
```

Bitmap indexes are commonly used in data warehousing environments, where large amounts of data are generally indexed on columns containing relatively few values (e.g., sales quarters, geographic regions, products, salespeople).

### Text indexes

If your database stores documents, you may need to allow users to search for words or phrases in the documents. You certainly don't want the server to peruse each document and scan for the desired text each time a search is requested, but traditional indexing strategies don't work for this situation. To handle this situation, MySQL, SQL Server, and Oracle Database include specialized indexing and search mechanisms for documents; both SQL Server and MySQL include what they call *full-text* indexes, and Oracle Database includes a powerful set of tools known as *Oracle Text*. Document searches are specialized enough that it is not practical to show an example, but it is useful to at least know what is available.

## How Indexes Are Used

Indexes are generally used by the server to quickly locate rows in a particular table, after which the server visits the associated table to extract the additional information requested by the user. Consider the following query:

```
mysql> SELECT customer_id, first_name, last_name
    -> FROM customer
    -> WHERE first_name LIKE 'S%' AND last_name LIKE 'P%';
+-------------+------------+-----------+
| customer_id | first_name | last_name |
+-------------+------------+-----------+
|          84 | SARA       | PERRY     |
|         197 | SUE        | PETERS    |
|         167 | SALLY      | PIERCE    |
+-------------+------------+-----------+
3 rows in set (0.00 sec)
```

For this query, the server can employ any of the following strategies:

- Scan all rows in the `customer` table.
- Use the index on the `last_name` column to find all customers whose last name starts with P; then visit each row of the `customer` table to find only rows whose first name starts with S.
- Use the index on the `last_name` and `first_name` columns to find all customers whose last name starts with P and whose first name starts with S.

The third choice seems to be the best option, since the index will yield all of the rows needed for the result set, without the need to revisit the table. But how do you know which of the three options will be utilized? To see how MySQL's query optimizer decides to execute the query, I use the `explain` statement to ask the server to show the execution plan for the query rather than executing the query:

```
mysql> EXPLAIN
    -> SELECT customer_id, first_name, last_name
    -> FROM customer
    -> WHERE first_name LIKE 'S%' AND last_name LIKE 'P%' \G;
*************************** 1. row ***************************
           id: 1
  select_type: SIMPLE
        table: customer
   partitions: NULL
         type: range
possible_keys: idx_last_name,idx_full_name
          key: idx_full_name
      key_len: 274
          ref: NULL
         rows: 28
     filtered: 11.11
        Extra: Using where; Using index
1 row in set, 1 warning (0.00 sec)
```

---

NOTE

Each database server includes tools to allow you to see how the query optimizer handles your SQL statement. SQL Server allows you to see an execution plan by issuing the statement `set showplan_text on` before running your SQL statement. Oracle Database includes the `explain plan` statement, which writes the execution plan to a special table called `plan_table`.

---

Looking at the query results, the `possible_keys` column tells you that the server could decide to use either the `idx_last_name` or the `idx_full_name` index, and the `key` column tells you that the `idx_full_name` index was chosen. Furthermore, the `type` column tells you that a range scan will be utilized, meaning that the database server will be looking for a range of values in the index, rather than expecting to retrieve a single row.

---

NOTE

The process that I just led you through is an example of query tuning. Tuning involves looking at an SQL statement and determining the resources available to the server to execute the statement. You can decide to modify the SQL statement, to adjust the database resources, or to do both in order to make a statement run more efficiently. Tuning is a detailed topic, and I strongly urge you to either read your server's tuning guide or pick up a good tuning book so that you can see all the different approaches available for your server.

---

## The Downside of Indexes

If indexes are so great, why not index everything? Well, the key to understanding why more indexes are not necessarily a good thing is to keep in mind that every index is a table (a special type of table but still a table). Therefore, every time a row is added to or removed from a table, all indexes on that table must be modified. When a row is updated, any indexes on the column or columns that were affected need to be modified as well. Therefore, the more indexes you have, the more work the server needs to do to keep all schema objects up-to-date, which tends to slow things down.

Indexes also require disk space as well as some amount of care from your administrators, so the best strategy is to add an index when a clear need arises. If you need an index for only special purposes, such as a monthly maintenance routine, you can always add the index, run the routine, and then drop the index until you need it again. In the case of data warehouses, where indexes are crucial during business hours as users run reports and ad hoc queries but are problematic when data is being loaded into the warehouse overnight, it is a common practice to drop the indexes before data is loaded and then recreate them before the warehouse opens for business.

In general, you should strive to have neither too many indexes nor too few. If you aren't sure how many indexes you should have, you can use this strategy as a default:

- Make sure all primary key columns are indexed (most servers automatically create unique indexes when you create primary key constraints). For multicolumn primary keys, consider building additional indexes on a subset of the primary key columns or

on all the primary key columns but in a different order than the primary key constraint definition.
- Build indexes on all columns that are referenced in foreign key constraints. Keep in mind that the server checks to make sure there are no child rows when a parent is deleted, so it must issue a query to search for a particular value in the column. If there's no index on the column, the entire table must be scanned.
- Index any columns that will frequently be used to retrieve data. Most date columns are good candidates, along with short (2- to 50-character) string columns.

After you have built your initial set of indexes, try to capture actual queries against your tables, look at the server's execution plan, and modify your indexing strategy to fit the most common access paths.

# Constraints

A constraint is simply a restriction placed on one or more columns of a table. There are several different types of constraints, including:

*Primary key constraints*

Identify the column or columns that guarantee uniqueness within a table

*Foreign key constraints*

Restrict one or more columns to contain only values found in another table's primary key columns (may also restrict the allowable values in other tables if `update cascade` or `delete cascade` rules are established)

*Unique constraints*

Restrict one or more columns to contain unique values within a table (primary key constraints are a special type of unique constraint)

*Check constraints*

Restrict the allowable values for a column

Without constraints, a database's consistency is suspect. For example, if the server allows you to change a customer's ID in the `customer` table without changing the same customer ID in the `rental` table, then you will end up with rental data that no longer points to valid customer records (known as *orphaned rows*). With primary and foreign key constraints in place, however, the server will either raise an error if an attempt is made

to modify or delete data that is referenced by other tables or propagate the changes to other tables for you (more on this shortly).

---

NOTE

If you want to use foreign key constraints with the MySQL server, you must use the InnoDB storage engine for your tables.

---

## Constraint Creation

Constraints are generally created at the same time as the associated table via the `create table` statement. To illustrate, here's an example from the schema generation script for the Sakila sample database:

```
CREATE TABLE customer (
  customer_id SMALLINT UNSIGNED NOT NULL AUTO_INCREMENT,
  store_id TINYINT UNSIGNED NOT NULL,
  first_name VARCHAR(45) NOT NULL,
  last_name VARCHAR(45) NOT NULL,
  email VARCHAR(50) DEFAULT NULL,
  address_id SMALLINT UNSIGNED NOT NULL,
  active BOOLEAN NOT NULL DEFAULT TRUE,
  create_date DATETIME NOT NULL,
  last_update TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    ON UPDATE CURRENT_TIMESTAMP,
  PRIMARY KEY (customer_id),
  KEY idx_fk_store_id (store_id),
  KEY idx_fk_address_id (address_id),
  KEY idx_last_name (last_name),
  CONSTRAINT fk_customer_address FOREIGN KEY (address_id)
    REFERENCES address (address_id) ON DELETE RESTRICT ON UPDATE CASCADE,
  CONSTRAINT fk_customer_store FOREIGN KEY (store_id)
    REFERENCES store (store_id) ON DELETE RESTRICT ON UPDATE CASCADE
)ENGINE=InnoDB DEFAULT CHARSET=utf8;
```

The `customer` table includes three constraints: one to specify that the `customer_id` column serves as the primary key for the table, and two more to specify that the `address_id` and `store_id` columns serve as foreign keys to the `address` and `store` table. Alternatively, you could create the `customer` table without foreign key constraints and add the foreign key constraints later via `alter table` statements:

```
ALTER TABLE customer
ADD CONSTRAINT fk_customer_address FOREIGN KEY (address_id)
REFERENCES address (address_id) ON DELETE RESTRICT ON UPDATE CASCADE;

ALTER TABLE customer
ADD CONSTRAINT fk_customer_store FOREIGN KEY (store_id)
REFERENCES store (store_id) ON DELETE RESTRICT ON UPDATE CASCADE;
```

Both of these statements include several `on` clauses:

- `on delete restrict`, which will cause the server to raise an error if a row is deleted in the parent table (`address` or `store`) that is referenced in the child table (`customer`)
- `on update cascade`, which will cause the server to propagate a change to the primary key value of a parent table (`address` or `store`) to the child table (`customer`)

The `on delete restrict` clause protects against orphaned records when rows are deleted from the parent table. To illustrate, let's pick a row in the `address` table and show the data from both the `address` and `customer` tables that share this value:

```
mysql> SELECT c.first_name, c.last_name, c.address_id, a.address
    -> FROM customer c
    ->   INNER JOIN address a
    ->   ON c.address_id = a.address_id
    -> WHERE a.address_id = 123;
+------------+-----------+------------+-----------------------------------+
| first_name | last_name | address_id | address                           |
+------------+-----------+------------+-----------------------------------+
| SHERRY     | MARSHALL  |        123 | 1987 Coacalco de Berriozbal Loop |
+------------+-----------+------------+-----------------------------------+
1 row in set (0.00 sec)
```

The results show that there is a single `customer` row (for Sherry Marshall) whose `address_id` column contains the value 123.

Here's what happens if you try to remove this row from the parent (`address`) table:

```
mysql> DELETE FROM address WHERE address_id = 123;
ERROR 1451 (23000): Cannot delete or update a parent row:
 a foreign key constraint fails (`sakila`.`customer`,
 CONSTRAINT `fk_customer_address` FOREIGN KEY (`address_id`)
```

```
  REFERENCES `address` (`address_id`)
  ON DELETE RESTRICT ON UPDATE CASCADE)
```

Because at least one row in the child table contains the value 123 in the `address_id` column, the `on delete restrict` clause of the foreign key constraint caused the statement to fail.

The `on update cascade` clause also protects against orphaned records when a primary key value is updated in the parent table using a different strategy. Here's what happens if you modify a value in the `address.address_id` column:

```
mysql> UPDATE address
    -> SET address_id = 9999
    -> WHERE address_id = 123;
Query OK, 1 row affected (0.37 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

The statement executed without error, and one row was modified. But what happened to Sherry Marshall's row in the `customer` table? Does it still point to address ID 123, which no longer exists? To find out, let's run the last query again, but substitute the new value 9999 for the previous value of 123:

```
mysql> SELECT c.first_name, c.last_name, c.address_id, a.address
    -> FROM customer c
    ->   INNER JOIN address a
    ->   ON c.address_id = a.address_id
    -> WHERE a.address_id = 9999;
+------------+-----------+------------+---------------------------------+
| first_name | last_name | address_id | address                         |
+------------+-----------+------------+---------------------------------+
| SHERRY     | MARSHALL  |       9999 | 1987 Coacalco de Berriozbal Loop |
+------------+-----------+------------+---------------------------------+
1 row in set (0.00 sec)
```

As you can see, the same results are returned as before (other than the new address ID value), which means that the value 9999 was automatically updated in the `customer` table. This is known as a *cascade*, and it's the second mechanism used to protect against orphaned rows.

Along with `restrict` and `cascade`, you can also choose `set null`, which will set the foreign key value to `null` in the child table when a row is deleted or updated in the parent table. Altogether, there are six different options to choose from when defining foreign key constraints:

- `on delete restrict`
- `on delete cascade`
- `on delete set null`
- `on update restrict`
- `on update cascade`
- `on update set null`

These are optional, so you can choose zero, one, or two (one `on delete` and one `on update`) of these when defining your foreign key constraints.

Finally, if you want to remove a primary or foreign key constraint, you can use the `alter table` statement again, except that you specify `drop` instead of `add`. While it is unusual to drop a primary key constraint, foreign key constraints are sometimes dropped during certain maintenance operations and then reestablished.

# Test Your Knowledge

Work through the following exercises to test your knowledge of indexes and constraints. When you're done, compare your solutions with those in Appendix B.

## Exercise 13-1

Generate an `alter table` statement for the `rental` table so that an error will be raised if a row having a value found in the `rental.customer_id` column is deleted from the `customer` table.

## Exercise 13-2

Generate a multicolumn index on the `payment` table that could be used by both of the following queries:

```
SELECT customer_id, payment_date, amount
FROM payment
WHERE payment_date > cast('2019-12-31 23:59:59' as datetime);
```

```
SELECT customer_id, payment_date, amount
FROM payment
WHERE payment_date > cast('2019-12-31 23:59:59' as datetime)
  AND amount < 5;
```

Previous chapter
< 12. Transactions

Next chapter
14. Views >